YAMLess Case Study — ngpos-infra-payments

YAMLess: Generating Azure Infrastructure from Code

Case Study — krogertechnology/ngpos-infra-payments

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given an **application** code repository, it produces a complete, policy-validated Azure Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

Applying YAMLess to ngpos-infra-payments

ngpos-infra-payments is the NGPOS Payments infrastructure repository. It contains hand-authored Terraform managing Azure Key Vault, Storage Account (with payment-specific blob containers), multi-environment workspaces (dev / test / stage / prod), and multi-region deployments (eastus2, centralus).

This repository is **not an application repo** — it contains no Dockerfile, no application source code, no package manifests, and no runtime dependencies. It is exactly the kind of output that YAMLess generates; running YAMLess against it is the equivalent of asking a code generator to process its own output.

Command run

```
yamless analyze /tmp/ngpos-infra-payments --no-llm
```

Profile detected

Field	Detected value	Confidence
Runtime	`unknown`	low
Shape	`unknown`	low
Port	`null`	—
Dockerfile	`false`	—
Database	`null`	—
Queue	`false`	—
Secrets	`[]`	—

No Terraform was generated. With all core fields at `low` confidence and `runtime: unknown`, there is no meaningful output yamless can produce. The static analyzer correctly found nothing because there is nothing application-related to find.

What the repo actually contains

```
terraform/
├── modules/
│   └── main_infra/
│       ├── main.tf       # Key Vault + Storage Account via Kroger
shared modules
│       └── variables.tf
└── workspaces/
    ├── dev/              # Storage containers: wic-settlement-file,
wic-hotlist-files,
    ├── test/             #   paw-bin-files, pinpad-config-files
    ├── stage/            # Multi-region: eastus2 + centralus
    └── prod/
```

The existing Terraform uses Kroger internal modules from `github.com/krogertechnology/ngpos-core-tfmodules`: - `ngpos-key-vault` — KV with AD group reader/writer roles and subnet allowlisting - `ngpos-azure-storage-account` — Storage Account with private endpoints and OAuth-only auth

This is well-structured, production-grade Terraform with multi-environment workspaces and a consistent CI/CD pipeline (plan on PR, apply on merge, drift detection cron, per-environment GitHub Actions workflows).

What this test case reveals about YAMLess

1. No guard against infra-only repos

What happened: YAMLess ran to completion and produced an all-`unknown` profile with no warning. A developer running this would see a confusing near-empty output with no explanation of why.

What should happen: YAMLess should detect infra-only repos early in the analysis phase and exit with a clear, helpful message:

```
✗ No application code detected in this repository.
  YAMLess generates Azure infrastructure from application repos
(Node.js, Python, Java).
  This repository appears to contain only Terraform/infrastructure
files.
  If the application lives in a different repo, run yamless against
that repo instead.
```

Detection heuristics (any of these with no Dockerfile / package.json / pom.xml / *.py): - `.tf` files at root or in `terraform/` directory - Absence of any runtime indicator after full static analysis - `runtime: unknown` AND `shape: unknown` AND no Dockerfile

Proposed fix (backlog #12): Add a pre-flight check in `runner.py` that detects infra-only repos and raises an `InfraOnlyRepoError` with a user-friendly message rather than returning a fully-unknown profile.

---

2. Opportunity: IaC-to-IaC comparison (future direction)

Observation: The NGPOS Payments team already has hand-authored infrastructure Terraform that was presumably written manually. If there is a companion application repo (e.g. a payments service that consumes this infrastructure), running YAMLess against the application repo would produce Terraform that could be compared against this hand-authored baseline.

This "diff" between YAMLess-generated and hand-authored Terraform could be a valuable audit tool: it would surface: - Resources YAMLess would provision that aren't in the hand-authored config (gaps in the manual work) - Resources in the hand-authored config that YAMLess doesn't generate yet (feature gaps in YAMLess) - Configuration drift between what the code implies and what infra was actually deployed

This is beyond the current scope of YAMLess but worth tracking as a future direction.

Summary

`ngpos-infra-payments` is an infra-only repository — it is precisely the type of output YAMLess produces, not the type of input it consumes. The static analyzer correctly found no application signals and returned an all-unknown profile.

No Terraform was generated — there is no application to generate infrastructure for.

The primary learning is that YAMLess needs an early guard for this case (backlog #12): detect infra-only repos and surface a clear, actionable error instead of a confusing empty profile. This is a one-function change in `runner.py`.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess